



Readiness Notification Patterns

There are two patterns for Readiness Notification:

- Pattern 1: Level-Triggered (poll, select, epoll)
 - You are notified as long as the file descriptor is ready
 - If you don't consume all data, you will be notified again immediately
 - You must read ALL available data or handle partial reads
 - Safer but more CPU intensive if not careful

```
Data arrives:      [=====]
Readiness:        ^^^^^^^^^^^^^^^^^ (continuous while data exists)
You read 1 byte:  ^^^^^^^^^^^^^^^^^ (still notified, data remains)
You read all:     (no more notification)
```

```
// Data arrives: 10 bytes in socket buffer
poll() returns POLLIN;

// You read only 5 bytes
recv(fd, buffer, 5, 0); // 5 bytes remain in kernel buffer

// Next poll() call (even without new data)
poll() returns POLLIN IMMEDIATELY; // Because 5 bytes still ready!

// If you don't consume everything, you get busy-loop
```

- Pattern 2: Edge-Triggered (epoll, kqueue)
 - You are notified ONLY ONCE when the state changes from "not ready" to "ready"
 - If you don't consume everything, you won't be notified again until NEW data arrives
 - You must read/write until EAGAIN / EWOULDBLOCK
 - More efficient (fewer system calls)
 - More dangerous if you don't consume everything

```

Data arrives:  [=====]
Readiness:    ^                (only at the moment of transition)
You read 1 byte:                (no notification, data still there!)
New data arrives: ^            (notified again because of transition)

```

```

// Set edge-triggered mode
ev.events = EPOLLIN | EPOLLET; // EPOLLET = Edge-Triggered

// Data arrives: 10 bytes
epoll_wait() returns this FD;

// You read only 5 bytes
recv(fd, buffer, 5, 0); // 5 bytes remain

// Next epoll_wait()
epoll_wait() DOES NOT return (blocking) // Because no STATE CHANGE

// Data remains in buffer but you're blocked!
// New data arrives (5 more bytes)
epoll_wait() returns; // Transition: 0→5 or 5→10

// You must read until EAGAIN to avoid missing data

```